

- Description

This module manages a vehicle fuel consumption monitoring system, ensuring that:

1. **Fuel efficiency is calculated based on distance traveled and fuel used.**
2. **If fuel efficiency falls below a threshold, a low-efficiency warning is triggered.**
3. **The system suggests fuel-saving driving adjustments when efficiency is poor.**
4. **A log of fuel consumption, efficiency, and warnings is maintained for analysis.**

This system helps improve fuel efficiency, reduce costs, and promote eco-friendly driving habits.

- Specification

module FuelEfficiencySystemEx

var distance, fuelUsed, efficiency, efficiencyThreshold: real

var consumptionLog: sequence of string

process recordTrip(distance: real, fuelUsed: real)

functional scenario form:

efficiency >= efficiencyThreshold && consumptionLog = consumptionLog $\oplus$
("Low Efficiency Warning: " + efficiency), Efficiency recorded successfully.

||

efficiency < efficiencyThreshold && consumptionLog = consumptionLog $\oplus$
("Efficiency Recorded: " + efficiency), Low efficiency warning triggered.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.AbstractTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.List;
/**
 * =====
 * c117_FuelEfficiencySystemEx
```

```

* -----
-----
* Goal:
*   - Expand the original FuelMonitor demo to paper-ready scale.
*   - Keep ALL original testcases (VT/FT/Z3/UVT) inside this file as c
omments
*       so you can enable any one by simply uncommenting.
*   - Default main() executes VT -> FT -> Z3 -> UVT sequentially (you
can
*       comment out any 3 groups to run only one).
*
* Modules:
*   1) Domain: VehicleTrip, FuelProfile (threshold), EfficiencyReport
*   2) Sensor: FuelSensor (input validation, trend, debounce warnings)
*   3) Service: EfficiencyService (km/L computation, classification)
*   4) Controller: EfficiencyController (recordTrip, print logs)
*   5) Logging: Event bus to Console + CSV (runs/c117_events.csv)
*   6) Alerts: Console / MockEmail / MockSMS by AlertHub
*   7) Recorder/Replay: CSV ops record & replay
*   8) Swing GUI: dashboard to try inputs & run scenarios
*   9) Compat: keep an API-
compatible wrapper of the original FuelMonitor
*
* Notes:
*   - For pedagogy: verbose comments, simple data structures, mock I/O
.
*   - For reproducibility: we keep your giant FT block exactly as prov
ided.
* =====
=====
*/
// ----- small utilities & shared types -----
-----

final class F117Console {
    private F117Console(){}
    static void info(String fmt, Object...args){ System.out.println("[I
NFO ] " + String.format(fmt, args)); }
    static void warn(String fmt, Object...args){ System.out.println("[W
ARN ] " + String.format(fmt, args)); }
    static void err (String fmt, Object...args){ System.err.println("[E
RROR] " + String.format(fmt, args)); }
}

final class F117Time {
    private F117Time(){}

```

```

        static String now(){ return LocalDateTime.now().format(DateTimeForm
atter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
    }
    final class F117Ids {
        private static long seq=0;
        static synchronized String next(String prefix){ return prefix + "-"
+ (++seq) + "-" + System.nanoTime(); }
    }
    enum F117Severity { INFO, WARNING, CRITICAL }
    enum F117EvtType { TRIP, INIT, ALERT, NOTE }
    // ----- Logging & events -----
    -----
    final class F117Event {
        final String id = F117Ids.next("EV");
        final String ts = F117Time.now();
        final String subject; // e.g., vehicleId or monitor name
        final F117EvtType type;
        final double distance; // km
        final double fuel; // liters
        final double eff; // km/L (if applicable)
        final String message;
        F117Event(String subject, F117EvtType type, double distance, double
fuel, double eff, String message){
            this.subject = subject;
            this.type = type;
            this.distance = distance;
            this.fuel = fuel;
            this.eff = eff;
            this.message = message;
        }
    }
    interface F117EventSink extends Closeable {
        void accept(F117Event e);
        default void close(){ /* no-op */ }
    }
    final class F117ConsoleSink implements F117EventSink {
        @Override public void accept(F117Event e){
            F117Console.info("%s | %-10s | %-
6s | dist=%10.2f | fuel=%8.2f | eff=%8.2f | %s",
                e.ts, e.subject, e.type.name(), e.distance, e.fuel, e.e
ff, e.message);
        }
    }
    final class F117CsvSink implements F117EventSink {

```

```

        private final BufferedWriter out;
        F117CsvSink(Path csv) throws IOException {
            Files.createDirectories(csv.getParent() == null ? Path.of(".") : csv.getParent());
            boolean exists = Files.exists(csv);
            out = Files.newBufferedWriter(csv, StandardCharsets.UTF_8,
                exists ? java.nio.file.StandardOpenOption.APPEND : java.
nio.file.StandardOpenOption.CREATE);
            if(!exists){
                out.write("ts,id,subject,type,distance,fuel,eff,message");
                out.newLine();
                out.flush();
            }
        }
        @Override public void accept(F117Event e){
            try{
                out.write(String.join(",",
                    e.ts, e.id, q(e.subject), e.type.name(),
                    String.valueOf(e.distance), String.valueOf(e.fuel),
String.valueOf(e.eff),
                    q(e.message)));
                out.newLine();
                out.flush();
            }catch (IOException ex){ F117Console.err("CSV write failed: %s"
, ex.getMessage()); }
        }
        private static String q(String s){ return "\"" + s.replace("\"", "\\\"")
+ "\""; }
        @Override public void close(){ try{ out.close(); }catch(Exception i
gnore){} }
    }
    final class F117EventLogger implements Closeable {
        private final List<F117EventSink> sinks = new ArrayList<>();
        void addSink(F117EventSink s){ sinks.add(s); }
        void log(F117Event e){ for(F117EventSink s: sinks) s.accept(e); }
        @Override public void close(){ for(F117EventSink s: sinks) try{ s.c
lose(); }catch(Exception ignore){} }
    }
    final class F117ViewBufferSink implements F117EventSink {
        private final List<F117Event> buf;
        F117ViewBufferSink(List<F117Event> buf){ this.buf = buf; }
        @Override public void accept(F117Event e){ buf.add(e); }
    }

```

```

// ----- alerts -----
-----
interface F117AlertChannel {
    void send(F117Severity sev, String title, String body);
}
final class F117ConsoleAlert implements F117AlertChannel {
    @Override public void send(F117Severity sev, String title, String body){
        F117Console.warn("ALERT (%s) %s - %s", sev, title, body);
    }
}
final class F117MockEmail implements F117AlertChannel {
    private final String to;
    F117MockEmail(String to){ this.to = to; }
    @Override public void send(F117Severity sev, String title, String body){
        F117Console.info("[MockEmail -> %s] [%s] %s\n%s", to, sev, title, body);
    }
}
final class F117MockSms implements F117AlertChannel {
    private final String to;
    F117MockSms(String to){ this.to = to; }
    @Override public void send(F117Severity sev, String title, String body){
        F117Console.info("[MockSMS -> %s] [%s] %s | %s", to, sev, title, body);
    }
}
final class F117AlertHub {
    private final List<F117AlertChannel> channels = new ArrayList<>();
    void add(F117AlertChannel c){ channels.add(c); }
    void notifyAll(F117Severity sev, String title, String body){
        for(F117AlertChannel c: channels) c.send(sev, title, body);
    }
}
// ----- domain -----
-----
final class FuelProfile {
    final double efficiencyThreshold; // km/L
    FuelProfile(double threshold){ this.efficiencyThreshold = threshold; }
}
final class VehicleTrip {

```

```

    final double distanceKm;
    final double fuelLiters;
    VehicleTrip(double distanceKm, double fuelLiters){
        this.distanceKm = distanceKm;
        this.fuelLiters = fuelLiters;
    }
}

final class EfficiencyReport {
    final double distance;
    final double fuel;
    final double efficiency; // km/L
    final boolean belowThreshold;
    final String message;
    EfficiencyReport(double distance, double fuel, double efficiency, boolean belowThreshold, String message){
        this.distance = distance;
        this.fuel = fuel;
        this.efficiency = efficiency;
        this.belowThreshold = belowThreshold;
        this.message = message;
    }
}

// ----- sensor -----
-----

final class FuelSensor {
    private final F117AlertHub alerts;
    private Double lastEff = null;
    private boolean warnedLowEff = false;
    FuelSensor(F117AlertHub alerts){ this.alerts = alerts; }
    void onTrip(EfficiencyReport rep, double threshold){
        // Trend note
        String trend;
        if(lastEff==null) trend = "n/a";
        else if(rep.efficiency > lastEff) trend = "improving";
        else if(rep.efficiency < lastEff) trend = "worsening";
        else trend = "stable";
        lastEff = rep.efficiency;
        if(rep.belowThreshold){
            if(!warnedLowEff){
                alerts.notifyAll(F117Severity.WARNING, "Low Fuel Efficiency",
                    String.format("Efficiency %.2f km/L below threshold %.2f (%s)", rep.efficiency, threshold, trend));
                warnedLowEff = true;
            }
        }
    }
}

```

```

        }
    } else {
        warnedLowEff = false;
    }
}
}

// ----- service -----
-----

final class EfficiencyService {
    private final FuelProfile profile;
    EfficiencyService(FuelProfile profile){ this.profile = profile; }
    EfficiencyReport evaluate(VehicleTrip t){
        if(t.fuelLiters <= 0 || t.distanceKm <= 0){
            return new EfficiencyReport(t.distanceKm, t.fuelLiters, 0.0
, true,
                "Invalid input: Distance and fuel consumption must
be greater than zero.");
        }
        double eff = t.distanceKm / t.fuelLiters;
        boolean below = eff < profile.efficiencyThreshold;
        String msg = String.format("Distance: %.1f km | Fuel Used: %.2f
L | Efficiency: %.2f km/L%s",
                t.distanceKm, t.fuelLiters, eff, below? " | WARNING: Lo
w Efficiency! Consider improving driving habits." : "");
        return new EfficiencyReport(t.distanceKm, t.fuelLiters, eff, be
low, msg);
    }
}

// ----- controller -----
-----

final class EfficiencyController {
    private final String subject; // e.g., "Fleet-A"
    private final EfficiencyService service;
    private final FuelSensor sensor;
    private final F117EventLogger logger;
    private final List<String> logLines = new ArrayList<>();
    EfficiencyController(String subject, EfficiencyService svc, FuelSen
sor sensor, F117EventLogger logger){
        this.subject = subject;
        this.service = svc;
        this.sensor = sensor;
        this.logger = logger;
        logger.log(new F117Event(subject, F117EvtType.INIT, 0,0,0, "Fue
l system initialized, threshold="+svcProfile().efficiencyThreshold));
    }
}

```

```

    }
    public void recordTrip(double distance, double fuel){
        EfficiencyReport rep = service.evaluate(new VehicleTrip(distance, fuel));
        System.out.println(rep.message);
        logLines.add(rep.message);
        // events
        logger.log(new F117Event(subject, F117EvtType.TRIP, rep.distance, rep.fuel, rep.efficiency, rep.message));
        // alerts via sensor
        sensor.onTrip(rep, svcProfile().efficiencyThreshold);
    }
    private FuelProfile svcProfile(){
        // tiny helper to access profile; we keep profile immutable in service
        try{
            var f = EfficiencyService.class.getDeclaredField("profile");
            f.setAccessible(true);
            return (FuelProfile) f.get(service);
        }catch(Exception e){ return new FuelProfile(15.0); }
    }
    public void printConsumptionLog(){
        System.out.println("\nFuel Consumption Log:");
        for(String s: logLines) System.out.println(s);
    }
}
// ----- Recorder/Replay -----
final class F117OpsRecorder implements Closeable {
    private final BufferedWriter out;
    F117OpsRecorder(Path file) throws IOException {
        Files.createDirectories(file.getParent()==null? Path.of("."): file.getParent());
        out = Files.newBufferedWriter(file, StandardCharsets.UTF_8,
            Files.exists(file)? java.nio.file.StandardOpenOption.APPEND : java.nio.file.StandardOpenOption.CREATE);
        if(Files.size(file)==0){
            out.write("ts,op,distance,fuel,note");
            out.newLine();
        }
    }
    void recordTrip(double distance, double fuel, String note) throws IOException {

```

```

        out.write(F117Time.now()+",TRIP,"+distance+", "+fuel+", "+q(note)
    );
    out.newLine(); out.flush();
}
private static String q(String s){ return "\"" + s.replace("\"","\\\"")
+ "\""; }
@Override public void close() throws IOException { out.close(); }
}
final class F117OpsReplayer {
    static void play(Path file, EfficiencyController ctrl) throws IOExc
eption {
        if(!Files.exists(file)){ F117Console.warn("Replay file missing:
%s", file); return; }
        List<String> lines = Files.readAllLines(file, StandardCharsets.
UTF_8);
        for(int i=1;i<lines.size();i++){
            String[] parts = split(lines.get(i));
            if(parts.length<5) continue;
            String op = parts[1];
            double dist = safeD(parts[2]);
            double fuel = safeD(parts[3]);
            if("TRIP".equals(op)) ctrl.recordTrip(dist, fuel);
        }
    }
    private static String[] split(String s){
        List<String> out = new ArrayList<>();
        boolean inQ=false;
        StringBuilder sb = new StringBuilder();
        for(int i=0;i<s.length();i++){
            char c = s.charAt(i);
            if(c==''){ inQ=!inQ; sb.append(c); }
            else if(c==',' && !inQ){ out.add(unq(sb.toString())); sb.se
tLength(0); }
            else sb.append(c);
        }
        out.add(unq(sb.toString()));
        return out.toArray(new String[0]);
    }
    private static String unq(String s){
        s = s.trim();
        if(s.startsWith("\"") && s.endsWith("\"")) return s.substring(1
, s.length()-1).replace("\\\"", "\"");
        return s;
    }
}

```

```

        private static double safeD(String s){ try{ return Double.parseDouble(s.trim()); }catch(Exception e){ return 0.0; } }
    }
    // ----- GUI -----
    -----
    final class F117GUI {
        private final JFrame frame = new JFrame("Fuel Efficiency Monitor");
        private final EfficiencyController ctrl;
        private final List<F117Event> viewBuf;
        private final F117EventTableModel tableModel;
        private final JLabel info = new JLabel();
        private final JTextField distField = new JTextField("120.0");
        private final JTextField fuelField = new JTextField("8.0");
        F117GUI(EfficiencyController ctrl, List<F117Event> buf){
            this.ctrl = ctrl; this.viewBuf = buf; this.tableModel = new F117EventTableModel(buf);
            init();
        }
        private void init(){
            frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
            frame.setSize(1080,700);
            frame.setLocationRelativeTo(null);
            JPanel root = new JPanel(new BorderLayout(10,10));
            root.setBorder(new EmptyBorder(10,10,10,10));
            JLabel title = new JLabel("Fleet: demo-car-01");
            title.setFont(title.getFont().deriveFont(Font.BOLD, 20f));
            info.setText("Enter distance (km) and fuel (L), then click Record Trip");
            JPanel top = new JPanel(new GridLayout(2,1,5,5));
            top.add(title); top.add(info);
            root.add(top, BorderLayout.NORTH);
            JTable table = new JTable(tableModel);
            table.setAutoCreateRowSorter(true);
            root.add(new JScrollPane(table), BorderLayout.CENTER);
            JPanel right = new JPanel();
            right.setLayout(new BoxLayout(right, BoxLayout.Y_AXIS));
            right.add(new JLabel("Distance (km):")); distField.setMaximumSize(new Dimension(200,28)); right.add(distField);
            right.add(Box.createVerticalStrut(6));
            right.add(new JLabel("Fuel (L):")); fuelField.setMaximumSize(new Dimension(200,28)); right.add(fuelField);
            right.add(Box.createVerticalStrut(8));
            JButton record = new JButton(new AbstractAction("Record Trip"){
                @Override public void actionPerformed(ActionEvent e){

```

```

        double d = pd(distField.getText(), 100);
        double f = pd(fuelField.getText(), 6);
        ctrl.recordTrip(d,f);
        tableModel.fireTableDataChanged();
    }
});
right.add(record);
right.add(Box.createVerticalStrut(10));
JButton vt = new JButton(new AbstractAction("Run VT"){
    @Override public void actionPerformed(ActionEvent e){ new F
117Scripts(ctrl,null).runVT(); tableModel.fireTableDataChanged(); }
});
JButton ft = new JButton(new AbstractAction("Run Fuzzer(120)"){
    @Override public void actionPerformed(ActionEvent e){ new F
117Scripts(ctrl,null).runVT(); tableModel.fireTableDataChanged(); }
});
JButton z3 = new JButton(new AbstractAction("Run Pseudo-Z3"){
    @Override public void actionPerformed(ActionEvent e){ new F
117Scripts(ctrl,null).runZ3(); tableModel.fireTableDataChanged(); }
});
JButton uvt = new JButton(new AbstractAction("Run UVT head"){
    @Override public void actionPerformed(ActionEvent e){ new F
117Scripts(ctrl,null).runUVTHead(); tableModel.fireTableDataChanged();
}

});
right.add(vt); right.add(Box.createVerticalStrut(5));
right.add(ft); right.add(Box.createVerticalStrut(5));
right.add(z3); right.add(Box.createVerticalStrut(5));
right.add(uvt);
root.add(right, BorderLayout.EAST);
frame.setContentPane(root);
}

private static double pd(String s, double def){ try{ return Double.
parseDouble(s.trim()); }catch(Exception e){ return def; } }
void show(){ SwingUtilities.invokeLater(() -> frame.setVisible(true
)); }

static final class F117EventTableModel extends AbstractTableModel {
    private final List<F117Event> data;
    private final String[] cols = { "Time", "ID", "Subject", "Type"
, "Distance", "Fuel", "Efficiency", "Message" };
    F117EventTableModel(List<F117Event> data){ this.data = data; }
    @Override public int getRowCount(){ return data.size(); }
    @Override public int getColumnCount(){ return cols.length; }
    @Override public String getColumnName(int c){ return cols[c]; }
}

```

```

@Override public Object getValueAt(int r, int c){
    F117Event e = data.get(r);
    return switch (c){
        case 0 -> e.ts;
        case 1 -> e.id;
        case 2 -> e.subject;
        case 3 -> e.type.name();
        case 4 -> e.distance;
        case 5 -> e.fuel;
        case 6 -> e.eff;
        case 7 -> e.message;
        default -> "";
    };
}
}
}
// ----- scripts/fuzz -----
-----
final class F117Scripts {
    private final EfficiencyController ctrl;
    private final F117OpsRecorder rec; // nullable
    F117Scripts(EfficiencyController ctrl, F117OpsRecorder rec){
        this.ctrl = ctrl; this.rec = rec;
    }
    private void step(double d, double f, String tag){
        ctrl.recordTrip(d,f);
        if(rec!=null) try{ rec.recordTrip(d,f, tag);}catch(IOException
ignore){}
    }
    void runVT(){
        double[][] arr = {
            {-399.37,479.28},
            {185.65,131.21},
            {-237.52,-144.81},
            {-276.69,480.34},
            {430.27,-475.26},
            {-185.11,323.64},
            {336.08,61.85},
            {81.00,262.77},
            {423.02,-53.50},
            {22.17,129.93},
            {-258.12,430.96},
            {-320.86,159.80},
            {-95.79,456.37},

```

{-361.48,-251.44},
{305.07,483.99},
{-408.17,-222.20},
{-274.78,-453.53},
{193.13,243.25},
{-200.49,397.34},
{-356.27,120.59},
{88.77,253.01},
{-211.50,-45.15},
{-332.76,454.70},
{-471.61,-251.61},
{-448.71,20.88},
{215.90,-227.64},
{-307.14,-289.79},
{250.62,471.65},
{-81.10,-464.02},
{139.46,58.82},
{293.08,388.58},
{60.36,-221.05},
{363.82,78.01},
{449.93,-411.57},
{-497.38,168.99},
{-275.05,-30.13},
{182.19,-32.25},
{-340.23,25.91},
{-134.46,-310.97},
{-443.19,240.56},
{-356.40,214.22},
{240.24,298.57},
{-65.38,-71.44},
{-331.92,-109.53},
{-377.65,225.58},
{-497.41,-127.34},
{35.23,441.17},
{-330.93,471.64},
{234.73,389.15},
{333.65,85.61},
{175.99,-240.40},
{441.78,-395.32},
{159.47,-447.02},
{-61.96,-412.06},
{187.01,-170.70},
{375.89,175.00},
{394.18,-468.78},

{487.58,-75.78},
{273.64,444.20},
{189.96,305.75},
{311.52,424.12},
{274.59,136.11},
{-125.33,2.10},
{427.86,247.89},
{464.34,-392.17},
{-273.54,435.47},
{-6.43,356.74},
{-454.04,308.86},
{215.02,170.34},
{30.43,-4.12},
{-442.96,394.63},
{-36.30,386.29},
{39.82,-92.78},
{245.87,355.27},
{-299.50,-466.62},
{-464.10,-63.60},
{188.00,58.41},
{407.45,-164.48},
{184.62,-336.48},
{-155.23,483.90},
{-359.17,-302.07},
{-114.45,277.38},
{-414.38,-473.13},
{368.83,90.54},
{291.62,-467.71},
{264.90,-392.46},
{278.58,364.75},
{-43.22,81.87},
{211.04,-100.30},
{-356.88,-363.49},
{454.10,341.97},
{327.98,-94.73},
{35.35,408.34},
{437.67,-209.76},
{-144.69,407.91},
{323.14,-489.44},
{194.79,-1.91},
{-457.85,286.00},
{-112.15,-480.38},
{-23.12,-208.37},

};

[illegible]

[illegible]

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]

[illegible][illegible][illegible][illegible][illegible][illegible][illegible][illegible]
$$\{-0.00, 0.00\},$$
$$\{-0.00, 0.00\},$$
 $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ [illegible][illegible][illegible]

[illegible][illegible]

```
{0.00,0.00},
{0.00,0.00},
{0.00,0.00},
{0.00,0.00},
{0.00,0.00},
{0.00,0.00},
{0.00,0.00},
{0.00,0.00},
{-
```

[illegible][illegible][illegible]
$$\{0.00, 0.00\},$$

[illegible]

[illegible] $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{-$ [illegible]

{779625120911999800000000000000000000000000000000000000}

[illegible] $\{-$ [illegible] $\{-$ [illegible] $\{-$ [illegible] $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{0.00, 0.00\},$ $\{-$ [illegible]

[illegible]

{-408.17, -222.20},
{-274.78, -453.53},
{193.13, 243.25},
{-200.49, 397.34},
{-356.27, 120.59},
{88.77, 253.01},
{-211.50, -45.15},
{-332.76, 454.70},
{-471.61, -251.61},
{-448.71, 20.88},
{215.90, -227.64},
{-307.14, -289.79},
{250.62, 471.65},
{-81.10, -464.02},
{139.46, 58.82},
{293.08, 388.58},
{60.36, -221.05},
{363.82, 78.01},
{449.93, -411.57},
{-497.38, 168.99},
{-275.05, -30.13},
{182.19, -32.25},
{-340.23, 25.91},
{-134.46, -310.97},
{-443.19, 240.56},
{-356.40, 214.22},
{240.24, 298.57},
{-65.38, -71.44},
{-331.92, -109.53},
{-377.65, 225.58},
{-497.41, -127.34},
{35.23, 441.17},
{-330.93, 471.64},
{234.73, 389.15},
{333.65, 85.61},
{175.99, -240.40},
{441.78, -395.32},
{159.47, -447.02},
{-61.96, -412.06},
{187.01, -170.70},
{375.89, 175.00},
{394.18, -468.78},
{487.58, -75.78},
{273.64, 444.20},

```
{189.96,305.75},
{311.52,424.12},
{274.59,136.11},
{-125.33,2.10},
{427.86,247.89},
{464.34,-392.17},
{-273.54,435.47},
{-6.43,356.74},
{-454.04,308.86},
{215.02,170.34},
{30.43,-4.12},
{-442.96,394.63},
{-36.30,386.29},
{39.82,-92.78},
{245.87,355.27},
{-299.50,-466.62},
{-464.10,-63.60},
{188.00,58.41},
{407.45,-164.48},
{184.62,-336.48},
{-155.23,483.90},
{-359.17,-302.07},
{-114.45,277.38},
{-414.38,-473.13},
{368.83,90.54},
{291.62,-467.71},
{264.90,-392.46},
{278.58,364.75},
{-43.22,81.87},
{211.04,-100.30},
{-356.88,-363.49},
{454.10,341.97},
{327.98,-94.73},
{35.35,408.34},
{437.67,-209.76},
{-144.69,407.91},
{323.14,-489.44},
{194.79,-1.91},
{-457.85,286.00},
{-112.15,-480.38},
{-23.12,-208.37},
};
for(double[] a: arr) step(a[0], a[1], "UVT");
}
```

```
void runZ3(){  
    double[][] arr = {  
        {-399.37,479.28},  
        {185.65,131.21},  
        {-237.52,-144.81},  
        {-276.69,480.34},  
        {430.27,-475.26},  
        {-185.11,323.64},  
        {336.08,61.85},  
        {81.00,262.77},  
        {423.02,-53.50},  
        {22.17,129.93},  
        {-258.12,430.96},  
        {-320.86,159.80},  
        {-95.79,456.37},  
        {-361.48,-251.44},  
        {305.07,483.99},  
        {-408.17,-222.20},  
        {-274.78,-453.53},  
        {193.13,243.25},  
        {-200.49,397.34},  
        {-356.27,120.59},  
        {88.77,253.01},  
        {-211.50,-45.15},  
        {-332.76,454.70},  
        {-471.61,-251.61},  
        {-448.71,20.88},  
        {215.90,-227.64},  
        {-307.14,-289.79},  
        {250.62,471.65},  
        {-81.10,-464.02},  
        {139.46,58.82},  
        {293.08,388.58},  
        {60.36,-221.05},  
        {363.82,78.01},  
        {449.93,-411.57},  
        {-497.38,168.99},  
        {-275.05,-30.13},  
        {182.19,-32.25},  
        {-340.23,25.91},  
        {-134.46,-310.97},  
        {-443.19,240.56},  
        {-356.40,214.22},
```

{240.24,298.57},
{-65.38,-71.44},
{-331.92,-109.53},
{-377.65,225.58},
{-497.41,-127.34},
{35.23,441.17},
{-330.93,471.64},
{234.73,389.15},
{333.65,85.61},
{175.99,-240.40},
{441.78,-395.32},
{159.47,-447.02},
{-61.96,-412.06},
{187.01,-170.70},
{375.89,175.00},
{394.18,-468.78},
{487.58,-75.78},
{273.64,444.20},
{189.96,305.75},
{311.52,424.12},
{274.59,136.11},
{-125.33,2.10},
{427.86,247.89},
{464.34,-392.17},
{-273.54,435.47},
{-6.43,356.74},
{-454.04,308.86},
{215.02,170.34},
{30.43,-4.12},
{-442.96,394.63},
{-36.30,386.29},
{39.82,-92.78},
{245.87,355.27},
{-299.50,-466.62},
{-464.10,-63.60},
{188.00,58.41},
{407.45,-164.48},
{184.62,-336.48},
{-155.23,483.90},
{-359.17,-302.07},
{-114.45,277.38},
{-414.38,-473.13},
{368.83,90.54},
{291.62,-467.71},

```

        {264.90,-392.46},
        {278.58,364.75},
        {-43.22,81.87},
        {211.04,-100.30},
        {-356.88,-363.49},
        {454.10,341.97},
        {327.98,-94.73},
        {35.35,408.34},
        {437.67,-209.76},
        {-144.69,407.91},
        {323.14,-489.44},
        {194.79,-1.91},
        {-457.85,286.00},
        {-112.15,-480.38},
        {-23.12,-208.37},
    };
    for(double[] a: arr) step(a[0], a[1], "Z3");
}
}
// ----- compatibility wrapper (original API) -----
-----
class FuelMonitor {
    private final EfficiencyController ctrl;
    FuelMonitor(double efficiencyThreshold){
        FuelProfile profile = new FuelProfile(efficiencyThreshold);
        F117EventLogger logger = new F117EventLogger();
        logger.addSink(new F117ConsoleSink());
        FuelSensor sensor = new FuelSensor(new F117AlertHub()); // no external alerts
        this.ctrl = new EfficiencyController("compat-monitor", new EfficiencyService(profile), sensor, logger);
    }
    public void recordTrip(double distance, double fuelUsed){ ctrl.recordTrip(distance, fuelUsed); }
    public void printConsumptionLog(){ ctrl.printConsumptionLog(); }
}
// ----- MAIN -----
-----
public class c117_FuelEfficiencySystemEx {
    // global view buffer for GUI/summary
    private static final List<F117Event> EVENT_VIEW = new ArrayList<>();
;
    private static void printCompactTail(){
        System.out.println("\n=== Compact Event Tail (last 80) ===");
    }
}

```

```

        int n = Math.min(80, EVENT_VIEW.size());
        int start = EVENT_VIEW.size() - n;
        for(int i=Math.max(0,start); i<EVENT_VIEW.size(); i++){
            F117Event e = EVENT_VIEW.get(i);
            System.out.printf("%s | %s | %s | dist=%.2f | fuel=%.2f | e
ff=%.2f | %s\n",
                            e.ts, e.subject, e.type, e.distance, e.fuel, e.eff,
e.message);
        }
        System.out.println("=== End ===");
    }

    public static void main(String[] args) {
        F117EventLogger logger = new F117EventLogger();
        logger.addSink(new F117ConsoleSink());
        logger.addSink(new F117ViewBufferSink(EVENT_VIEW));
        try{
            logger.addSink(new F117CsvSink(Path.of("runs", "c117_events
.csv"))));
        }catch(IOException e){
            F117Console.warn("CSV sink disabled: %s", e.getMessage());
        }
        F117AlertHub alerts = new F117AlertHub();
        alerts.add(new F117ConsoleAlert());
        alerts.add(new F117MockEmail("ops@example.com"));
        alerts.add(new F117MockSms("+81-90-0000-0000"));
        FuelProfile profile = new FuelProfile(15.0);
        EfficiencyService svc = new EfficiencyService(profile);
        FuelSensor sensor = new FuelSensor(alerts);
        EfficiencyController ctrl = new EfficiencyController("fleet-
demo", svc, sensor, logger);
        F117Scripts scripts = new F117Scripts(ctrl, null);
        System.out.println("\n===== [VT Test Sequence] ===
=====");
        //      scripts.runVT();
        System.out.println("=====
=====\\n");
        System.out.println("\n===== [FT Fuzz Test Sequence]
=====");
        scripts.runFT();
        System.out.println("=====
=====\\n");
        System.out.println("\n===== [Z3 Pseudo Test Sequenc
e] =====");
        //      scripts.runZ3();

```

```

        System.out.println("=====
=====\\n");
        System.out.println("\\n===== [UVT Extended Test Sequ
ence] =====");
//        scripts.runUVTHed();
        System.out.println("=====
=====\\n");
        printCompactTail();
        // new F117GUI(ctrl, EVENT_VIEW).show();
        // try { F117OpsReplayer.play(Path.of("runs", "my_trips.csv"), c
        trL); } catch (IOException ignore) {}
    }
}

```

- Coverage report

1. Test case number: 100
2. V-Method testing branch coverage: 90%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c117_FuelEfficiencySystemEx.java	31.0 %	27	60	87
> EfficiencyController	0.0 %	0	2	2
> F117Scripts	37.5 %	3	5	8
> F117CsvSink	50.0 %	3	3	6
> F117EventLogger	50.0 %	2	2	4
> EfficiencyService	75.0 %	6	2	8
▼ FuelSensor	90.0 %	9	1	10
▲ onTrip(EfficiencyReport, double)	90.0 %	9	1	10
▲ FuelSensor(F117AlertHub)		0	0	0
> c117_FuelEfficiencySystemEx	100.0 %	2	0	2
> F117AlertHub	100.0 %	2	0	2
> EfficiencyReport		0	0	0
> F117Console		0	0	0
> F117ConsoleAlert		0	0	0
> F117ConsoleSink		0	0	0
> F117Event		0	0	0
> F117EvtType		0	0	0
> F117Ids		0	0	0
> F117MockEmail		0	0	0
> F117MockSms		0	0	0
> F117Severity		0	0	0
> F117Time		0	0	0
> F117ViewBufferSink		0	0	0

3. Vibration testing instruction coverage: 97.6%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c117_FuelEfficiencySystemEx.java		49.2 %	1,077	1,110	2,187
> F117Scripts		56.0 %	177	139	316
> F117EventLogger		63.8 %	30	17	47
> F117Console		66.7 %	14	7	21
> F117CsvSink		75.8 %	100	32	132
> EfficiencyController		78.4 %	91	25	116
> F117Ids		80.0 %	12	3	15
> EfficiencyService		86.9 %	73	11	84
> c117_FuelEfficiencySystemEx		93.5 %	188	13	201
▼ FuelSensor		97.6 %	81	2	83
▲ onTrip(EfficiencyReport, double)		97.2 %	69	2	71
▲ FuelSensor(F117AlertHub)		100.0 %	12	0	12
> EfficiencyReport		100.0 %	18	0	18
> F117AlertHub		100.0 %	32	0	32
> F117ConsoleAlert		100.0 %	20	0	20
> F117ConsoleSink		100.0 %	47	0	47
> F117Event		100.0 %	28	0	28
> F117EvtType		100.0 %	44	0	44
> F117MockEmail		100.0 %	28	0	28
> F117MockSms		100.0 %	28	0	28
> F117Severity		100.0 %	34	0	34
> F117Time		100.0 %	5	0	5
> F117ViewBufferSink		100.0 %	12	0	12

4. Fuzz testing branch coverage: 70%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c117_FuelEfficiencySystemEx.java		25.3 %	22	65	87
> EfficiencyController		0.0 %	0	2	2
> EfficiencyService		37.5 %	3	5	8
> F117Scripts		37.5 %	3	5	8
> F117CsvSink		50.0 %	3	3	6
> F117EventLogger		50.0 %	2	2	4
▼ FuelSensor		70.0 %	7	3	10
▲ onTrip(EfficiencyReport, double)		70.0 %	7	3	10
▲ FuelSensor(F117AlertHub)			0	0	0
> c117_FuelEfficiencySystemEx		100.0 %	2	0	2
> F117AlertHub		100.0 %	2	0	2
> EfficiencyReport			0	0	0
> F117Console			0	0	0
> F117ConsoleAlert			0	0	0
> F117ConsoleSink			0	0	0
> F117Event			0	0	0
> F117EvtType			0	0	0
> F117Ids			0	0	0
> F117MockEmail			0	0	0
> F117MockSms			0	0	0
> F117Severity			0	0	0
> F117Time			0	0	0
> F117ViewBufferSink			0	0	0

5. Fuzz testing instruction coverage: 89.2%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c117_FuelEfficiencySystemEx.java		44.2 %	920	1,163	2,083
> EfficiencyService		32.1 %	27	57	84
> F117Scripts		34.4 %	73	139	212
> F117EventLogger		63.8 %	30	17	47
> F117Console		66.7 %	14	7	21
> F117CsvSink		75.8 %	100	32	132
> EfficiencyController		78.4 %	91	25	116
> F117Ids		80.0 %	12	3	15
▼ FuelSensor		89.2 %	74	9	83
▲ onTrip(EfficiencyReport, double)		87.3 %	62	9	71
▲ FuelSensor(F117AlertHub)		100.0 %	12	0	12
> c117_FuelEfficiencySystemEx		93.5 %	188	13	201
> EfficiencyReport		100.0 %	18	0	18
> F117AlertHub		100.0 %	32	0	32
> F117ConsoleAlert		100.0 %	20	0	20
> F117ConsoleSink		100.0 %	47	0	47
> F117Event		100.0 %	28	0	28
> F117EvtType		100.0 %	44	0	44
> F117MockEmail		100.0 %	28	0	28
> F117MockSms		100.0 %	28	0	28
> F117Severity		100.0 %	34	0	34
> F117Time		100.0 %	5	0	5
> F117ViewBufferSink		100.0 %	12	0	12

6. Z3 branch coverage: 90%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c117_FuelEfficiencySystemEx.java		32.1 %	26	55	81
> EfficiencyController		0.0 %	0	2	2
> F117Scripts		30.0 %	3	7	10
> F117CsvSink		50.0 %	3	3	6
> F117EventLogger		50.0 %	2	2	4
> EfficiencyService		62.5 %	5	3	8
▼ FuelSensor		90.0 %	9	1	10
▲ onTrip(EfficiencyReport, double)		90.0 %	9	1	10
▲ FuelSensor(F117AlertHub)			0	0	0
> c117_FuelEfficiencySystemEx		100.0 %	2	0	2
> F117AlertHub		100.0 %	2	0	2
> EfficiencyReport			0	0	0
> F117Console			0	0	0
> F117ConsoleAlert			0	0	0
> F117ConsoleSink			0	0	0
> F117Event			0	0	0
> F117EvtType			0	0	0
> F117Ids			0	0	0
> F117MockEmail			0	0	0
> F117MockSms			0	0	0
> F117Severity			0	0	0
> F117Time			0	0	0
> F117ViewBufferSink			0	0	0

7. Z3 instruction coverage: 96.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c117_FuelEfficiencySystemEx.java	31.6 %	2,255	4,881	7,136
> F117Scripts	25.2 %	1,347	3,995	5,342
> F117EventLogger	63.8 %	30	17	47
> F117Console	66.7 %	14	7	21
> F117CsvSink	75.8 %	100	32	132
> EfficiencyController	78.4 %	91	25	116
> F117Ids	80.0 %	12	3	15
> c117_FuelEfficiencySystemEx	93.5 %	188	13	201
▼ FuelSensor	96.4 %	80	3	83
▲ onTrip(EfficiencyReport, double)	95.8 %	68	3	71
▲ FuelSensor(F117AlertHub)	100.0 %	12	0	12
> EfficiencyService	97.6 %	82	2	84
> EfficiencyReport	100.0 %	18	0	18
> F117AlertHub	100.0 %	32	0	32
> F117ConsoleAlert	100.0 %	20	0	20
> F117ConsoleSink	100.0 %	47	0	47
> F117Event	100.0 %	28	0	28
> F117EvtType	100.0 %	44	0	44
> F117MockEmail	100.0 %	28	0	28
> F117MockSms	100.0 %	28	0	28
> F117Severity	100.0 %	34	0	34
> F117Time	100.0 %	5	0	5
> F117ViewBufferSink	100.0 %	12	0	12

8. Unimproved V-method branch coverage: 90%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c117_FuelEfficiencySystemEx.java	31.6 %	2,255	4,881	7,136
> F117Scripts	25.2 %	1,347	3,995	5,342
> F117EventLogger	63.8 %	30	17	47
> F117Console	66.7 %	14	7	21
> F117CsvSink	75.8 %	100	32	132
> EfficiencyController	78.4 %	91	25	116
> F117Ids	80.0 %	12	3	15
> c117_FuelEfficiencySystemEx	93.5 %	188	13	201
▼ FuelSensor	96.4 %	80	3	83
▲ onTrip(EfficiencyReport, double)	95.8 %	68	3	71
▲ FuelSensor(F117AlertHub)	100.0 %	12	0	12
> EfficiencyService	97.6 %	82	2	84
> EfficiencyReport	100.0 %	18	0	18
> F117AlertHub	100.0 %	32	0	32
> F117ConsoleAlert	100.0 %	20	0	20
> F117ConsoleSink	100.0 %	47	0	47
> F117Event	100.0 %	28	0	28
> F117EvtType	100.0 %	44	0	44
> F117MockEmail	100.0 %	28	0	28
> F117MockSms	100.0 %	28	0	28
> F117Severity	100.0 %	34	0	34
> F117Time	100.0 %	5	0	5
> F117ViewBufferSink	100.0 %	12	0	12

9. Unimproved V-method instruction coverage: 96.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼  c117_FuelEfficiencySystemEx.java	 31.6 %	2,255	4,881	7,136
>  F117Scripts	 25.2 %	1,347	3,995	5,342
>  F117EventLogger	63.8 %	30	17	47
>  F117Console	66.7 %	14	7	21
>  F117CsvSink	75.8 %	100	32	132
>  EfficiencyController	78.4 %	91	25	116
>  F117Ids	80.0 %	12	3	15
>  c117_FuelEfficiencySystemEx	93.5 %	188	13	201
▼  FuelSensor	96.4 %	80	3	83
▲ onTrip(EfficiencyReport, double)	 95.8 %	68	3	71
▲ FuelSensor(F117AlertHub)	 100.0 %	12	0	12
>  EfficiencyService	97.6 %	82	2	84
>  EfficiencyReport	100.0 %	18	0	18
>  F117AlertHub	100.0 %	32	0	32
>  F117ConsoleAlert	100.0 %	20	0	20
>  F117ConsoleSink	100.0 %	47	0	47
>  F117Event	100.0 %	28	0	28
>  F117EvtType	100.0 %	44	0	44
>  F117MockEmail	100.0 %	28	0	28
>  F117MockSms	100.0 %	28	0	28
>  F117Severity	100.0 %	34	0	34
>  F117Time	100.0 %	5	0	5
>  F117ViewBufferSink	100.0 %	12	0	12